

Kyber / ML-KEM Coding Session — Continuation Brief

Generated to hand off this session to a fresh Claude Code session. This document replaces and extends the earlier `Kyber_Project_Brief.pdf` with everything built, learned, and understood since.

1. Dissertation Context

MSc Cryptography dissertation: **Post-Quantum Cryptography — Investigating Lattice-Based Algorithms and a Quantum-Resistant Secure Channel**. Student: Brian Julian Patiño Florez (20012183). Module CT7P01. The project implements ML-KEM (FIPS 203, formerly Kyber) from scratch in Python, argues for lattice-based cryptography as a future-proof replacement for RSA/ECDH, and investigates pairing the resulting shared secret with AES-256 to build a full quantum-resistant secure channel. **ML-KEM-1024 is the headline/demonstration variant** ($k=4$, $\eta_1=2$, $\eta_2=2$, $du=11$, $dv=5$), chosen deliberately for the largest security margin, motivated in the dissertation by ongoing cryptanalytic erosion of lattice problem margins.

2. Author Background & Working Style — READ THIS FIRST

Python experience is weak/near-beginner. Strong grasp of the underlying cryptographic theory (LWE, MLWE, polynomial rings, the full Kyber algorithm, FO transform, sampling, NTT conceptually, compression). Comes from a Maple/mathematics background, not software engineering. Wants code checkable against hand calculations, not trusted blindly.

Non-negotiable working rules (established through explicit correction):

- **Discuss before coding, every time — no exceptions.** Even when the user gives a clear, detailed specification, do NOT write code immediately. Explain the plan/logic in chat first, get explicit confirmation, THEN write. The user has explicitly corrected this before ("you assume to code straight away") and it is a standing instruction, not a one-off preference.
- **One tiny piece at a time.** Show one line or one small concept, explain it, wait for confirmation, then move to the next. Do not batch multiple new concepts into one explanation.
- **When the user says they don't understand, narrow down rather than re-explain broadly.** Ask which specific line/word/step is unclear instead of repeating the same explanation a different way. Repeating the same style of explanation that already failed rarely helps.
- **Concrete real numbers beat abstraction.** Hand-traceable worked examples with real numbers (run through the actual code, not just asserted) are consistently the most effective teaching tool in this session. Toy/small parameters first, then scale up.
- **Dense visual diagrams have backfired.** A 4x4 grid (16 cells) showing a multiplication table made things WORSE, not better. Simple diagrams (2-3 example boxes with arrows) work; anything busier does not. When in doubt, prefer a plain sequential numbered-steps text trace over a diagram.
- **The user likes renaming variables and adding their own phrasing as code comments.** When they explain something in their own words and it's correct (or corrected), add it to the file as a comment in close to their own phrasing.
- **Independent verification matters a lot to this user.** Don't just assert correctness — actually check it (e.g. `poly_mult` was verified against 2000 random trials against an independently-written reference calculation; the CBD lookup table was exhaustively checked against all 256 possible byte values). This builds trust and is worth doing proactively.

- User sometimes asks to delete/revert everything and restart when overwhelmed (happened once early on with the polynomial file). Comply without pushback — it's a legitimate coping strategy for this user, not a sign anything is broken.
- Casual typing style, occasional typos, sometimes answers "si" for yes. Treat short/ambiguous replies charitably but confirm scope on anything destructive or large before proceeding.
- User is a genuinely careful, persistent learner who catches real subtleties (spotted an accumulator-vs-reference distinction unprompted; independently proposed the CBD lookup-table optimization; asked sophisticated 'why not just do X' questions that were often genuinely good engineering questions, not just confusion).

3. All Files Created — Full Inventory

All files live in `~/Desktop/Kyber codes/` on the user's Mac.

toy_integer_kyber_3x3.py — COMPLETE, VERIFIED WORKING

The very first build: a full toy KEM using plain integers (no polynomials). $q=97$ (originally 17, changed after a real decryption failure was diagnosed live — see bugs section). $k=3$, 3×3 matrices. Contains: Bob's KeyGen (A , S , e , b), Alice's Encapsulate (r , e_{one} , e_{two} , m , m_{encoded} , A_t , b_t , u , v), Bob's Decapsulate (S_t , w , $\text{zero_check/half_check}$ decoding, $m_{\text{recovered}}$). Verified across 8 consecutive runs, all matched. This file is the reference "shape" that every later file mirrors line-for-line, just swapping operators.

kyber_toy_code_explained.pdf / kyber_toy_alice_explained.pdf

Diagram + plain-English PDFs (built with reportlab) covering Bob's KeyGen and Alice's Encapsulate sections of the integer version, matching the box/arrow flowchart style established early on.

toy_poly_kyber_3x3.py — COMPLETE, VERIFIED WORKING

The polynomial version, using REAL FIPS 203 constants: $n=256$, $q=3329$, $k=4$ (matches ML-KEM-1024's k). Contains `poly_add`, `poly_sub`, `poly_mult` (schoolbook multiplication with the $x^n=-1$ fold-and-subtract trick, $O(n^2)$), and the full KeyGen/Encapsulate/Decapsulate mirroring the integer version exactly, with `poly_add/poly_mult` standing in for $+/*$. Currently still uses `random.choice([-1,0,1])` for small values ($\eta \sim 1$ equivalent), NOT yet real FIPS-accurate CBD sampling — that upgrade is in progress in separate files (see below) but not yet merged back in. Verified across 8 consecutive runs, all matched.

verify_poly_mult.py

Independent verification script: runs 2000 random polynomial pairs through `poly_mult` and cross-checks against a differently-implemented reference (uses `exponent//n` and `exponent%n` directly instead of the subtract-once fold). All 2000 matched. Deliberately written differently from `poly_mult` itself so it's a genuine independent check.

kyber_toy_poly_explained.pdf

Diagram + explanation PDF for `poly_add`, `poly_sub`, `poly_mult` (the "boxes" multiply-then-fold visual), plus a summary page explaining that KeyGen/Encaps/Decaps are structurally identical to the integer version.

toy_hash_kyber_3x3.py — IN PROGRESS, KeyGen ONLY so far

The target file for the real hashing/FO-transform version. $n=256$, $q=3329$, $k=4$. Currently contains ONLY Bob's KeyGen section, copied unchanged from `toy_poly_kyber_3x3.py` (KeyGen doesn't involve the FO transform, only Encapsulate/Decapsulate do). Verified working. **Nothing else has been merged into this file yet** — all the real hashing/CBD work below has been built and verified in separate teaching files first, per the user's explicit "discuss and understand each piece first" request.

hash_example.py — COMPLETE, VERIFIED WORKING (most fully-realized piece)

The main teaching file for the FO transform's hashing mechanism. Full working pipeline: $m=[1,1]$, $A=[[4,4],[20,4]]$, $b=[5,1]$ (small illustrative example values) $\rightarrow$ combined = `str(m)+str(A)+str(b)` $\rightarrow$ `.encode()` to bytes $\rightarrow$ `hashlib.sha3_512(...).digest()` (fixed 64-byte "seed") $\rightarrow$ `hashlib.shake_256(seed).digest(1152)` ("big_output", exactly the calculated total bytes needed for $r+e_{\text{one}}+e_{\text{two}}$ at $k=4$, $\eta=2$) $\rightarrow$ sliced into `r_bytes[0:512]`, `e_one_bytes[512:1024]`, `e_two_bytes[1024:1152]` $\rightarrow$ a full 256-entry CBD_TABLE is embedded

directly in the file (not imported) → a function `poly_builder(poly_bytes, table)` converts one byte-chunk into one polynomial via table lookups (dict lookup + `.extend()`) → loops build `r` (4 polynomials) and `e_one` (4 polynomials) via 128-byte slices each (`i*bytes_per_poly` to `(i+1)*bytes_per_poly`), `e_two` built directly from its one slice with no loop. Produces real, correctly small (-2 to 2) coefficients. This file demonstrates the ENTIRE real hash-to-noise pipeline end to end and matches FIPS 203's actual approach (not a simplified stand-in), except for the one acknowledged simplification noted below.

`cbd_sampling_example.py` — COMPLETE, VERIFIED WORKING

Contains `bytes_to_bits(B)` (extracts 8 bits per byte via `%2/#2`, LSB-first — i.e. REVERSED order compared to normally-written binary) and `sample_poly_cbd(bits, eta, num_coefficients, verbose=True)` (the direct x-y calculation method, with a verbose flag added later to suppress per-coefficient printing when called many times for real use). Also contains `build_cbd_table(eta)` and `lookup_coefficients(byte_sequence, table)` — the table-building alternative, proposed by the USER as an optimization idea, confirmed valid specifically because `eta=2` divides evenly into 8 bits/byte (would NOT work cleanly for `eta=3`, used by ML-KEM-512's `eta1`). The `__main__` block demonstrates both methods on the same 4 bytes ([13,201,7,255]) and confirms they match exactly.

`cbd_table_256.py` — COMPLETE, EXHAUSTIVELY VERIFIED

All 256 possible byte values (0-255), written out as an explicit literal Python dictionary (`CBD_TABLE`), each mapped to its 2 real coefficients for `eta=2` — generated programmatically then written as static data per the user's explicit request to see every case hand-writable/inspectable, not hidden behind a runtime loop. Exhaustively verified: all 256 entries individually re-checked against a fresh independent calculation, 256/256 correct. This exact table is also pasted directly (not imported) into `hash_example.py` per the user's later request to keep that file self-contained.

4. Concepts Covered This Session (in roughly the order taught)

Python fundamentals

- Variables, comments (`#`), `print()`, lists, indexing (`list[i]`), for loops including nested loops, `range()` (starts at 0, stops one short)
- Functions (`def`), return values, the "dot means an action belonging to that thing" pattern (`.append()`, `.encode()`, `.digest()`, `.extend()`)
- Modular arithmetic (`%`) and integer/floor division (`//`) — thoroughly distinguished from real division
- String concatenation with `+` (different meaning than numeric `+`), `str()`, `repr()`, f-strings
- Dictionaries and dict lookup (`table[key]`) — new vs earlier list-only indexing
- `.append()` (adds one item) vs `.extend()` (adds multiple items from a list) — explicitly contrasted
- List/bytes slicing (`[start:end]`)
- Importing your own functions from another file you wrote (`from X import Y`), as opposed to only built-in libraries
- Variable scope: everything is global at top level in this project (no functions wrapping the main script), contrasted with Maple's local/proc distinction
- Accumulator pattern (running total, e.g. `dot_product`) explicitly distinguished from plain reference/alias (e.g. `row = A[row_num]`, `b_t = b`) and from one-shot calculation — user spotted this distinction themselves

Linear algebra → code

- Vectors as lists, matrices as list-of-lists
- Dot product built from scratch as an accumulator loop
- Matrix transpose: real code needed (row/col genuinely swap) vs vector "transpose" being a complete no-op in code (`b_t = b`, `S_t = S`) since a flat list has no orientation to flip — this distinction was reached via extensive back-and-forth and eventually clicked

LWE / Module-LWE correctness

- $A \cdot S + e = b$, and why S/e (and later $r/e_1/e_2$) MUST be small — demonstrated as a REAL live bug: S was initially generated full-range like A , which broke decryption completely (noise term S_{t,e_one} swamped the message). Fixed by switching S to `random.choice([-1,0,1])`. This was a genuinely powerful teaching moment and is referenced repeatedly later in the session.
- Why q doesn't need to be prime for the arithmetic to work (only division needs it) but prime avoids zero-divisor structural weakness — later directly paralleled with why x^n+1 needs n to be a power of 2 (irreducibility, not "does it run")
- The cancellation trick: why $w = v - S_t \cdot u$ recovers the message (A/S terms cancel, leaving message + small noise)

Toy KEM build (integer version)

- Full KeyGen → Encapsulate → Decapsulate cycle, hand-verified against the user's own pre-existing hand-worked notes (`Kyber_From_Scratch.md`)
- Message encoding: bit → 0 or $q/2$ via the "multiply as switch" trick $(q//2)*m$
- Decoding/rounding bug: naive `abs(w-0)` vs `abs(w-q//2)` check failed to account for modular wraparound (a value near $q-1$ is actually close to 0 going the other way around). Fixed with `zero_check = min(abs(w-0), abs(w-q))`

- Correct terminology: Encrypt/Decrypt (PKE, one layer) vs Encapsulate/Decapsulate (KEM, the outer layer) — explicitly clarified early on since the user asked for the proper terms

Polynomials

- A polynomial is a list of coefficients (position = power of x)
- Why lists don't "defeat the purpose" of using polynomials — the power comes from the multiplication CLOSURE property (a ring), not from the storage format, which is identical to a vector's
- Schoolbook multiplication: multiply every pair of terms into "boxes" ($\text{raw}[i+j]$), then fold anything at position n or higher back down using $x^n = -1$ (subtract instead of add, since it's a negative wraparound) — taught via a "boxes" analogy that worked well
- Why n must be a power of 2 for the NTT specifically (not for basic ring arithmetic, which works for any n) — and why x^{n+1} must be irreducible ($n = \text{power of } 2$) for security, directly paralleling the earlier q -primality discussion
- Scaling from toy $n=4, q=11$ (hand-verified against the user's own notes) up to real $n=256, q=3329$

FIPS 203 specifics (pulled directly from the actual standard PDF)

- Fixed constants: $n=256$, $q=3329 (=2^8 \cdot 13 + 1, \text{ prime})$
- Full Table 2: ML-KEM-512 ($k=2, \eta_1=3, \eta_2=2, du=10, dv=4$), ML-KEM-768 ($k=3, \eta_1=2, \eta_2=2, du=10, dv=4$), ML-KEM-1024 ($k=4, \eta_1=2, \eta_2=2, du=11, dv=5$)
- Security categories ($1/3/5 \approx \text{AES-128/192/256-equivalent hardness}$) and decapsulation failure rates (Table 1)
- Hash functions: $H=\text{SHA3-256}$ (hashes the public key), $G=\text{SHA3-512}$ (produces TWO 32-byte outputs at once, used for K and r), $J=\text{SHAKE256 fixed-32-byte}$ (fallback/implicit-rejection key), $\text{PRF}_{\eta}=\text{SHAKE256 variable-length}$ (expands seeds into SamplePolyCBD input), $\text{XOF}=\text{SHAKE128 variable-length}$ (expands a public seed into matrix A so A never needs to be transmitted directly)
- η_1/η_2 control noise "smallness" (CBD bound); du/dv control ciphertext compression (NOT yet built in this project)

The Fujisaki-Okamoto (FO) transform

- WHY it's needed: defends against chosen-ciphertext attacks — without it, an attacker could send tampered ciphertexts and learn information from Bob's responses
- HOW it works: derive the encryption randomness FROM the message (hash m together with the public key) instead of picking it independently → makes encryption deterministic for a given m → on decrypt, re-derive and re-encrypt to check the ciphertext matches → if it doesn't match, return a pseudorandom fallback key instead of an error (so a tampered ciphertext can't be distinguished from a legitimate one from the outside)
- Full algorithm walkthrough with every hash call location identified: KeyGen computes+stores $H(ek)$ once; Encapsulate computes $H(ek)$ itself then $G(m||H(ek)) \rightarrow K, r\text{-seed}$; Decapsulate re-does $G(m'||H(ek)) \rightarrow K', r'\text{-seed}$, recomputes u'/v' , compares to received (u, v) , falls back to $J(z||c)$ on mismatch

Real hashing mechanics

- `.encode()` converts text to bytes via a FIXED LOOKUP TABLE (ASCII, which UTF-8 matches exactly for normal characters) — not calculated, just looked up, same as every computer
- Hash functions only accept bytes, never plain text — this IS the reason `.encode()` is needed at all
- SHA3-256/SHA3-512: FIXED output size (the number in the name = output size in bits)
- SHAKE128/SHAKE256: VARIABLE output size, chosen by the caller via `.digest(N)` — the number in THEIR name means security strength level instead (like AES-128 vs AES-256), not output size, since output size is flexible
- SHAKE output is one continuous stream — asking for more bytes just continues the same stream (demonstrated: the first 8 bytes of a 32-byte request exactly matched a standalone 8-byte request)
- Why one hash call isn't enough material: SHA3-512 caps at 64 bytes, but `r+e_one+e_two` together need 1152 bytes (at $k=4$, $\eta=2$) — solved with a two-stage approach (`sha3_512` for a fixed seed, then `shake_256(seed).digest(1152)` to expand it) — this mirrors FIPS 203's real G-then-PRF structure

SamplePolyCBD (real FIPS 203 sampling, not the random.choice simplification)

- `bytes_to_bits`: extracts each byte's 8 bits via repeated `%2` (get bit) / `#2` (shift down) — produces bits in REVERSED order compared to normally-written binary (LSB first) — this was a major, carefully-worked-through confusion point, resolved via multiple hand traces (bytes 6, 200, 151, 97, 115, 7, 153 all traced by hand)
- For each coefficient: sum the next η bits (x), sum the η bits after that (y), coefficient = $x - y \rightarrow$ naturally small ($-\eta$ to $+\eta$) and bell-curve shaped (extremes require all bits to align, which is rare)
- Why raw bytes (0-255) can't be used directly as noise — directly recalled the earlier "S was too big" bug as the reason
- Why a single hash isn't enough bytes for everything needed → why SHAKE (not SHA3) is the right tool for that specific step
- The user independently proposed precomputing a 256-entry lookup table (byte → its coefficients) instead of recalculating each time — confirmed mathematically valid specifically for $\eta=2$ (divides evenly into 8 bits), confirmed it does NOT generalize to $\eta=3$ (ML-KEM-512's $\eta=1$), built and exhaustively verified

5. Bugs Found & Fixed Live (valuable context, don't reintroduce)

- **S generated full-range instead of small** (toy_integer_kyber_3x3.py, early on): used `random.randint(0,q-1)` like A instead of `random.choice([-1,0,1])` like e. Caused complete, reproducible decryption failure. This was diagnosed live and used as a major teaching moment about why LWE secrets must be small — do not repeat this mistake in any future variable.
- **Decoding didn't handle modular wraparound**: a plain `abs(w-0)` vs `abs(w-q//2)` check misclassifies values near $q-1$ (which are actually close to 0 going the other way around the modulus). Fixed with `zero_check = min(abs(w-0), abs(w-q))`.
- **VS Code "invalid argument to create terminal" error**: caused by VS Code having opened a single file instead of the project folder. Fixed via File → Open Folder → select the Kyber codes folder (not the file directly).

6. Exact Current Position & Next Steps

hash_example.py is the most complete, fully-verified demonstration of the real hashing-to-noise pipeline. **toy_hash_kyber_3x3.py** (the intended final destination) currently only has Bob's KeyGen section — none of the hashing/CBD work has been merged into it yet.

Immediate next steps, in order, still to do:

- Merge the **hash_example.py** pipeline into **toy_hash_kyber_3x3.py** for Alice's Encapsulate section: `m` (now `n` random bits, one per coefficient), `m_encoded`, real hash-derived `r/e_one/e_two` (replacing `random.choice`), `A_t`, `u`, `v` — following the exact discuss-first, one-piece-at-a-time pattern used throughout this session
- Build Bob's Decapsulate re-check (the actual FO transform security step, not yet implemented anywhere): after recovering `m_recovered`, Bob re-hashes `m_recovered+A+b`, re-derives `r/e_one/e_two` the same way, recomputes `u/v`, and compares to the actual received (`u,v`)
- Implement the fallback/"implicit rejection" behaviour on mismatch (return a different pseudorandom value instead of an error)
- Create diagrams + PDF for this new hashing/FO-transform section, matching the established style of the other explanation PDFs, once the code itself is finished
- **PINNED FOR LATER** (explicitly deferred, not forgotten): generating matrix A from a small seed via a SHAKE128-style XOF, instead of raw `random.randint` — discussed as a real space-saving technique real Kyber uses (transmit a 32-byte seed instead of the whole matrix), distinct from the FO-transform work and agreed to revisit "at the end of this python file"

Suggested prompt to open the new session with:

"I'm continuing a from-scratch Python implementation of ML-KEM (Kyber) for my MSc Cryptography dissertation. I have weak Python skills but strong grasp of the underlying maths. Please read the attached session brief PDF in full before doing anything else, then continue exactly where it leaves off: merging the hashing/CBD pipeline from hash_example.py into toy_hash_kyber_3x3.py's Alice/Bob sections, one small piece at a time, discussing each piece with me before writing any code — do not write code until I've confirmed I understand the plan for that specific piece."